Travaux Pratiques 1 — Introduction à la programmation de la carte UCA

Compte rendu

NOM Prénom — NOM Prénom · Groupe : ……

16 octobre 2026

Introduction

Ce TP sert à prendre en main la carte **UCA** (RFThings, microcontrôleur ATmega328PB). On la programme en C/C++ avec l'Arduino IDE. On utilise la LED bleue et le port série (UART) en sortie, puis en entrée. On lit ensuite les capteurs intégrés : SHTC3 (température et humidité), LTR-303A (lumière) et KXTJ3-1057 (accéléromètre). Enfin, on pilote les 21 LEDs RGB WS2812. La dernière partie combine ces éléments dans plusieurs programmes autonomes.

Tous les programmes sont écrits en **C++ orienté objet**. Chaque composant de la carte est représenté par une classe (fichiers `.h` / `.cpp`), et `setup()` / `loop()` se trouvent dans `main.cpp` (voir la section « Architecture C++ »). Ils ont été compilés pour la cible **RFThings UCA – version 3.9 et plus récente (ATmega328PB)** sans erreur ni dépassement mémoire (voir le tableau en annexe).

Architecture C++

Le code de l'énoncé est écrit dans le style procédural d'Arduino : variables globales, `#define` et appels directs aux bibliothèques. Nous l'avons réécrit en **C++ orienté objet** :

- **Une classe par composant de la carte**, qui *encapsule* la bibliothèque du constructeur. `main.cpp` ne manipule donc que des objets simples, comme `capteur.lire(mesure)` ou `bandeau.remplir(CRGB::Red)`.
- **`constexpr`** remplace `#define` pour les constantes. Elles sont typées et respectent la portée.
- **Espace de noms anonyme** (`namespace { … }`) : les objets globaux restent privés au fichier `main.cpp`.
- **Structures** (`MesureTempHum`, `MesureLumiere`, `Acceleration`) pour regrouper les valeurs d'une mesure. Elles sont passées **par référence** (`lire(MesureLumiere& mesure)`), et la fonction renvoie `bool` pour signaler une erreur de lecture.
- **`enum class`** pour des états typés (ex. `Inclinaison::Droite`).
- **Constructeurs avec liste d'initialisation**, mot-clé **`explicit`**, méthodes **`const`**, **`static_cast`** et boucle « range-for » (`for (CRGB& led : leds_)`).

Chaque dossier de projet contient `main.cpp`, les classes qu'il utilise et un fichier `.ino` presque vide. Ce `.ino` est obligatoire, car l'Arduino IDE ne sait ouvrir un projet que s'il contient un `.ino` portant le nom du dossier.

Classe	Rôle
`Led`	LED simple : `allumer()`, `eteindre()`, `basculer()`
`Minuterie`	Temporisation non bloquante avec `millis()` : `estEcoulee()`
`LecteurNombre`	Lecture non bloquante d'un nombre de plusieurs chiffres sur le port série
`CapteurTempHum`	SHTC3 : température (°C) et humidité (%HR)
`CapteurLumiere`	LTR-303 : canaux bruts et calcul des lux
`Accelerometre`	KXTJ3-1057 : accélération X, Y, Z en g
`BandeauLed`	21 LEDs WS2812 via FastLED
`Alerte`	Alerte IoT déclenchée sur front montant

Remarque : la bibliothèque KXTJ3 définit une macro `getName` qui entre en conflit avec FastLED dès que les deux sont incluses dans le même fichier. Le fichier `Accelerometre.h` supprime donc cette macro avec `#undef getName`.

Code des classes

Led

```cpp
#pragma once
#include <Arduino.h>

// Pilote une LED simple branchée sur une broche numérique.
class Led {
public:
  explicit Led(uint8_t broche);

  void begin();          // configure la broche en sortie
  void allumer();
  void eteindre();
  void basculer();       // allumée -> éteinte, éteinte -> allumée
  bool estAllumee() const;

private:
  uint8_t broche_;
  bool allumee_;
};
```

```cpp
#include "Led.h"

Led::Led(uint8_t broche) : broche_(broche), allumee_(false) {}
```

```cpp
void Led::begin() {
  pinMode(broche_, OUTPUT);
  eteindre();
}

void Led::allumer() {
  digitalWrite(broche_, HIGH);
  allumee_ = true;
}

void Led::eteindre() {
  digitalWrite(broche_, LOW);
  allumee_ = false;
}

void Led::basculer() {
  if (allumee_) {
    eteindre();
  } else {
    allumer();
  }
}

bool Led::estAllumee() const {
  return allumee_;
}
```

Minuterie

```cpp
#pragma once
#include <Arduino.h>

// Minuterie non bloquante basée sur millis().
// estEcoulee() renvoie true une seule fois par période.
class Minuterie {
public:
  explicit Minuterie(unsigned long periodeMs);

  bool estEcoulee();
  void setPeriode(unsigned long periodeMs);
  unsigned long periode() const;

private:
  unsigned long periodeMs_;
  unsigned long precedent_;
};
```

```cpp
#include "Minuterie.h"

Minuterie::Minuterie(unsigned long periodeMs) : periodeMs_(periodeMs),
precedent_(0) {}
```

```
bool Minuterie::estEcoulee() {
  const unsigned long maintenant = millis();
  // Soustraction non signée : reste correcte lors du débordement de
millis() (~49,7 jours)
  if (maintenant - precedent_ >= periodeMs_) {
    precedent_ = maintenant;
    return true;
  }
  return false;
}

void Minuterie::setPeriode(unsigned long periodeMs) {
  periodeMs_ = periodeMs;
}

unsigned long Minuterie::periode() const {
  return periodeMs_;
}
```

LecteurNombre

```
#pragma once
#include <Arduino.h>

// Lit de façon non bloquante un nombre entier positif saisi sur un
flux série
// (plusieurs chiffres, validé par la touche Entrée).
class LecteurNombre {
public:
  explicit LecteurNombre(Stream& flux);

  // Renvoie true quand un nombre complet a été reçu ; il est alors
écrit dans 'valeur'.
  bool lire(unsigned long& valeur);

private:
  Stream& flux_;
  unsigned long accumulateur_;
  bool chiffreRecu_;
};

#include "LecteurNombre.h"

LecteurNombre::LecteurNombre(Stream& flux) : flux_(flux),
accumulateur_(0), chiffreRecu_(false) {}

bool LecteurNombre::lire(unsigned long& valeur) {
  while (flux_.available() > 0) {
    const char c = static_cast<char>(flux_.read());
```

```
    if (c >= '0' && c <= '9') {
      accumulateur_ = accumulateur_ * 10 + static_cast<unsigned
long>(c - '0');  // ASCII -> chiffre
      chiffreRecu_ = true;
    } else if ((c == '\n' || c == '\r') && chiffreRecu_) {
      valeur = accumulateur_;
      accumulateur_ = 0;
      chiffreRecu_ = false;
      return true;
    }
    // Les autres caractères sont ignorés
  }
  return false;
}
```

CapteurTempHum

```
#pragma once
#include <Arduino.h>
#include <Wire.h>
#include "SHTSensor.h"

struct MesureTempHum {
  float temperature;  // °C
  float humidite;     // %HR
};

// Capteur de température et d'humidité SHTC3 (I2C, adresse 0x70).
class CapteurTempHum {
public:
  bool begin();
  bool lire(MesureTempHum& mesure);

private:
  SHTSensor sht_;
};
```

```
#include "CapteurTempHum.h"

bool CapteurTempHum::begin() {
  Wire.begin();
  return sht_.init();
}

bool CapteurTempHum::lire(MesureTempHum& mesure) {
  if (!sht_.readSample()) {
    return false;
  }
  mesure.temperature = sht_.getTemperature();
  mesure.humidite = sht_.getHumidity();
```

```
  return true;
}
```

CapteurLumiere

```
#pragma once
#include <Arduino.h>
#include "Adafruit_LTR329_LTR303.h"

struct MesureLumiere {
  uint16_t visibleIr;   // canal 0 : visible + infrarouge (comptes
bruts)
  uint16_t infrarouge;  // canal 1 : infrarouge seul (comptes bruts)
  float lux;            // éclairement calculé
};

// Capteur de lumière LTR-303A (I2C, adresse 0x29), gain x1.
class CapteurLumiere {
public:
  bool begin(ltr329_integrationtime_t integration =
LTR3XX_INTEGTIME_100,
             ltr329_measurerate_t frequence = LTR3XX_MEASRATE_500);

  bool nouvelleDonnee();
  bool lire(MesureLumiere& mesure);

private:
  static float integrationEnMs(ltr329_integrationtime_t integration);
  float calculerLux(uint16_t ch0, uint16_t ch1) const;

  Adafruit_LTR303 ltr_;
  float integrationMs_ = 100.0f;
};

#include "CapteurLumiere.h"

bool CapteurLumiere::begin(ltr329_integrationtime_t integration,
ltr329_measurerate_t frequence) {
  if (!ltr_.begin()) {
    return false;
  }
  ltr_.setGain(LTR3XX_GAIN_1);
  ltr_.setIntegrationTime(integration);
  ltr_.setMeasurementRate(frequence);
  integrationMs_ = integrationEnMs(integration);
  return true;
}

bool CapteurLumiere::nouvelleDonnee() {
  return ltr_.newDataAvailable();
}
```

```cpp
bool CapteurLumiere::lire(MesureLumiere& mesure) {
  if (!ltr_.readBothChannels(mesure.visibleIr, mesure.infrarouge)) {
    return false;
  }
  mesure.lux = calculerLux(mesure.visibleIr, mesure.infrarouge);
  return true;
}

float CapteurLumiere::integrationEnMs(ltr329_integrationtime_t
integration) {
  switch (integration) {
    case LTR3XX_INTEGTIME_50:  return 50.0f;
    case LTR3XX_INTEGTIME_150: return 150.0f;
    case LTR3XX_INTEGTIME_200: return 200.0f;
    case LTR3XX_INTEGTIME_250: return 250.0f;
    case LTR3XX_INTEGTIME_300: return 300.0f;
    case LTR3XX_INTEGTIME_350: return 350.0f;
    case LTR3XX_INTEGTIME_400: return 400.0f;
    default:                   return 100.0f;
  }
}

// Formule de la datasheet LTR-303 (gain x1) : dépend du rapport IR /
(visible + IR)
float CapteurLumiere::calculerLux(uint16_t ch0, uint16_t ch1) const {
  const float somme = static_cast<float>(ch0) + ch1;
  if (somme == 0.0f) {
    return 0.0f;
  }
  const float ratio = ch1 / somme;
  float lux;
  if (ratio < 0.45f) {
    lux = 1.7743f * ch0 + 1.1059f * ch1;
  } else if (ratio < 0.64f) {
    lux = 4.2785f * ch0 - 1.9548f * ch1;
  } else if (ratio < 0.85f) {
    lux = 0.5926f * ch0 + 0.1185f * ch1;
  } else {
    lux = 0.0f;
  }
  return lux / (integrationMs_ / 100.0f);  // correction du temps
d'intégration
}
```

Accelerometre

```cpp
#pragma once
#include <Arduino.h>
#include "kxtj3-1057.h"
```

```
// La bibliothèque KXTJ3 définit une macro getName() qui casse FastLED
: on la supprime.
#ifdef getName
#undef getName
#endif

struct Acceleration {
  float x;  // en g
  float y;
  float z;
};

// Accéléromètre 3 axes KXTJ3-1057 (I2C, adresse 0x0E ou 0x0F).
class Accelerometre {
public:
  explicit Accelerometre(uint8_t adresse = 0x0E);

  // frequence en Hz, plage en g (2, 4, 8 ou 16)
  bool begin(float frequence, uint8_t plage, bool hauteResolution =
false);
  void veille(bool active);
  Acceleration lire();

private:
  KXTJ3 imu_;
};

#include "Accelerometre.h"

Accelerometre::Accelerometre(uint8_t adresse) : imu_(adresse) {}

bool Accelerometre::begin(float frequence, uint8_t plage, bool
hauteResolution) {
  return imu_.begin(frequence, plage, hauteResolution) == IMU_SUCCESS;
}

void Accelerometre::veille(bool active) {
  imu_.standby(active);
}

Acceleration Accelerometre::lire() {
  return Acceleration{imu_.axisAccel(X), imu_.axisAccel(Y),
imu_.axisAccel(Z)};
}
```

BandeauLed

```
#pragma once
#include <Arduino.h>
#include <FastLED.h>
```

```cpp
// Les 21 LEDs RGB WS2812 de la carte UCA (ligne de données sur D4).
class BandeauLed {
public:
  static constexpr uint8_t NB_LEDS = 21;
  static constexpr uint8_t BROCHE_DONNEES = 4;

  void begin(uint8_t luminosite = 84);
  void definir(uint8_t index, const CRGB& couleur);
  void remplir(const CRGB& couleur);
  void attenuer(uint8_t facteur);  // multiplie chaque LED par
facteur/256
  void afficher();

private:
  CRGB leds_[NB_LEDS];
};

#include "BandeauLed.h"

void BandeauLed::begin(uint8_t luminosite) {
  // Si les couleurs sont inversées sur la carte, remplacer RGB par
GRB
  FastLED.addLeds<WS2812, BROCHE_DONNEES, RGB>(leds_, NB_LEDS);
  FastLED.setBrightness(luminosite);
  remplir(CRGB::Black);
  afficher();
}

void BandeauLed::definir(uint8_t index, const CRGB& couleur) {
  if (index < NB_LEDS) {
    leds_[index] = couleur;
  }
}

void BandeauLed::remplir(const CRGB& couleur) {
  fill_solid(leds_, NB_LEDS, couleur);
}

void BandeauLed::attenuer(uint8_t facteur) {
  for (CRGB& led : leds_) {
    led.nscale8(facteur);
  }
}

void BandeauLed::afficher() {
  FastLED.show();
}
```

Alerte

```
#pragma once
#include <Arduino.h>

// Alerte IoT : s'affiche une seule fois, au moment où sa condition
devient vraie.
// Format : "Alerte : <type> à <millis> ms"
class Alerte {
public:
  explicit Alerte(const char* type);

  void verifier(bool condition, Print& sortie);

private:
  const char* type_;
  bool active_;
};

#include "Alerte.h"

Alerte::Alerte(const char* type) : type_(type), active_(false) {}

void Alerte::verifier(bool condition, Print& sortie) {
  if (condition && !active_) {  // front montant : faux -> vrai
    sortie.print("Alerte : ");
    sortie.print(type_);
    sortie.print(" à ");
    sortie.print(millis());
    sortie.println(" ms");
  }
  active_ = condition;
}
```

1. Installation et configuration de l'Arduino IDE

Étape	Action réalisée
1.1	Installation de l'Arduino IDE 2.x (Windows 10/11 64 bits, ou archive ZIP si l'on n'a pas les droits administrateur).
1.2	*File → Preferences → Additional boards manager URLs* : `https://rfthings.github.io/ArduinoBoardManagerJSON/package_rfthings-avr_index.json`, puis installation du package **RFThings AVR Boards by RFThings Vietnam** (v0.1.1-rc6) dans le gestionnaire de cartes.
1.3	Installation du pilote **CH340** (puce de

Étape	Action réalisée
	conversion USB ↔ série de la carte).
1.4	*Tools* → *Board* → *RFThings AVR Boards* → *RFThings UCA*, *Board Version : 3.9 and newer (ATmega328PB)*, puis choix du port COM de la carte.
1.5	Installation de la bibliothèque **FastLED** (Library Manager).
1.6	Test avec *File* → *Examples* → *01.Basics* → *Blink* : la LED bleue clignote.

Question : à quoi sert le pilote CH340 ? L'ATmega328PB n'a pas de port USB natif. La puce CH340C convertit l'USB en liaison série (UART). Le PC voit alors un port COM virtuel, utilisé pour téléverser le programme (via le bootloader) et pour le moniteur série. Sans ce pilote, aucun port COM n'apparaît dans *Tools* → *Port*.

Question : pourquoi choisir la « Board Version » ? Les cartes UCA 3.8 et antérieures utilisent un ATmega328P, les 3.9 et suivantes un **ATmega328PB**. Ces deux microcontrôleurs n'ont pas exactement les mêmes périphériques ni le même brochage. Le compilateur doit donc connaître le bon modèle (`-mmcu`) et le bon fichier de brochage (*variant*).

Dans le *variant* UCA, `LED_BUILTIN` est défini à la broche **13**. C'est la LED bleue de la carte.

2. Programmation Arduino

Interface de l'IDE

N°	Élément	Rôle
1	Vérifier	Compile le sketch
2	Téléverser	Compile puis programme la carte
3	Débogage	Débogage pas à pas (non disponible sur AVR sans sonde)
4	Sélecteur carte/port	Choix de la carte et du port COM
5	Traceur série	Trace graphiquement les valeurs reçues
6	Moniteur série	Affiche le texte envoyé par `Serial.print()`
7–11	Barre latérale	Carnet de croquis, gestionnaire de cartes, de bibliothèques, débogueur, recherche

N°	Élément	Rôle
12	Éditeur	Code source C/C++ (`.ino`)
13	Zone de messages	Résultat de compilation et de téléversement

Structure d'un sketch (Blink)

- `setup()` est exécutée **une seule fois**, à la mise sous tension ou au reset. Elle sert aux initialisations. Ici, `pinMode(LED_BUILTIN, OUTPUT)` configure la broche 13 en sortie.
- `loop()` est une **boucle infinie** : son contenu est répété sans fin.
- `digitalWrite(broche, HIGH/LOW)` met la broche à 1 (5 V ou 3,3 V) ou à 0 (0 V).
- `delay(ms)` bloque le programme pendant le nombre de millisecondes donné.

```
// Étape 1.6 — Test de clignotement de la LED bleue (LED_BUILTIN = D13)
#include <Arduino.h>
#include "Led.h"

namespace {
constexpr unsigned long DUREE_MS = 1000;
Led ledBleue(LED_BUILTIN);
}  // namespace

void setup() {
  ledBleue.begin();
}

void loop() {
  ledBleue.allumer();
  delay(DUREE_MS);
  ledBleue.eteindre();
  delay(DUREE_MS);
}
```

Résultat : la LED est allumée 1 s puis éteinte 1 s. Sa période est donc de 2 s, soit une fréquence de 0,5 Hz.

Étape 2.1 — Sortie série (Blink_Mod)

```
// Étape 2.1 — Sortie série : l'état de la LED est envoyé sur l'UART (9600 bauds)
#include <Arduino.h>
#include "Led.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 9600;
constexpr unsigned long DUREE_MS = 1000;
Led ledBleue(LED_BUILTIN);
}  // namespace
```

```
void setup() {
  Serial.begin(DEBIT_BAUDS);
  ledBleue.begin();
  Serial.println("Initialisation OK");
}

void loop() {
  ledBleue.allumer();
  Serial.println(1);  // visible dans le moniteur et le traceur série
  delay(DUREE_MS);

  ledBleue.eteindre();
  Serial.println(0);
  delay(DUREE_MS);
}
```

- `Serial.begin(9600)` initialise l'UART à **9600 bauds** (bits par seconde). Une trame 8N1 fait 10 bits par caractère, ce qui donne environ 960 caractères/s.
- `Serial.println(x)` envoie la valeur en **texte ASCII**, suivie de `\r` (ASCII 13) et `\n` (ASCII 10).
- Le **moniteur série** affiche « Initialisation OK » puis alterne `1` et `0` chaque seconde. Le **traceur série** affiche un signal carré entre 0 et 1, de période 2 s.

Question : pourquoi le moniteur doit-il être à 9600 bauds ? L'UART est asynchrone : il n'y a pas de fil d'horloge. L'émetteur et le récepteur doivent donc utiliser le même débit. Sinon, le récepteur échantillonne les bits au mauvais moment et affiche des caractères incohérents.

Étape 2.2 — Entrée série

```
// Étape 2.2 — Entrée série : l'utilisateur saisit le délai de
clignotement (0..9 s)
#include <Arduino.h>
#include "Led.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 9600;
Led ledBleue(LED_BUILTIN);
unsigned long delaiMs = 1000;

// Attend un chiffre valide sur le port série et le renvoie (0..9)
uint8_t demanderChiffre() {
  for (;;) {
    Serial.println("Please enter the blink delay (0..9) in (s): ");
    while (!Serial.available()) {
      // attente active d'un caractère
    }
    const char c = static_cast<char>(Serial.read());
    while (Serial.available()) {
      Serial.read();  // vide le buffer (\r, \n...)
```

```
    }
    if (c >= '0' && c <= '9') {
      return static_cast<uint8_t>(c - '0');  // conversion ASCII ->
nombre
    }
    Serial.println("Entree invalide, recommencez.");
  }
}
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  ledBleue.begin();

  delaiMs = demanderChiffre() * 1000UL;  // secondes -> millisecondes
  Serial.print("Delai = ");
  Serial.print(delaiMs);
  Serial.println(" ms");
}

void loop() {
  ledBleue.allumer();
  delay(delaiMs);
  ledBleue.eteindre();
  delay(delaiMs);
}
```

- `Serial.available()` renvoie le **nombre d'octets** en attente dans le buffer de réception. La valeur est 0 si le buffer est vide. *Remarque* : c'est `Serial.read()` qui renvoie −1 quand aucun octet n'est disponible, pas `available()` comme l'indique l'énoncé.
- `while(!Serial.available());` est une **attente active** : on reste bloqué tant que l'utilisateur n'a rien envoyé.
- `Serial.read() - '0'` : le caractère reçu est un code ASCII. Par exemple, `'5'` vaut 53 et `'0'` vaut 48, donc 53 − 48 = **5**. Comme les chiffres `'0'`…`'9'` ont des codes consécutifs (48…57), cette soustraction convertit n'importe quel chiffre en sa valeur numérique.
- Par rapport à l'énoncé, on a ajouté une **vérification de l'entrée** : si le caractère n'est pas entre `'0'` et `'9'`, on redemande. On vide aussi le buffer pour supprimer les `\r` ou `\n` ajoutés par le moniteur selon l'option « Nouvelle ligne ». Sans cette vérification, une lettre comme `'a'` (97) donnerait 49, soit un délai de 49 s.

Question : comment lire un nombre de plusieurs chiffres (ex. 12) ? On lit **tous** les caractères disponibles dans une boucle `while (Serial.available())`. On construit le nombre avec `nombre = nombre × 10 + chiffre` et on le valide à la réception de la fin de ligne. Cette version lit la saisie **pendant** que la LED clignote (lecture dans `loop()`, clignotement non bloquant avec `millis()`) :

```
// Étape 2.2 (suite) — Lecture d'un nombre de plusieurs chiffres
pendant que la LED clignote.
```

```ino
// Taper un délai en millisecondes puis Entrée (moniteur série à 9600
bauds, "Nouvelle ligne").
#include <Arduino.h>
#include "Led.h"
#include "Minuterie.h"
#include "LecteurNombre.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 9600;
Led ledBleue(LED_BUILTIN);
Minuterie clignotement(1000);
LecteurNombre lecteur(Serial);
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  ledBleue.begin();
  Serial.println("Entrez un delai en ms puis Entree :");
}

void loop() {
  unsigned long nouveauDelai;
  if (lecteur.lire(nouveauDelai)) {
    clignotement.setPeriode(nouveauDelai);
    Serial.print("Nouveau delai : ");
    Serial.print(nouveauDelai);
    Serial.println(" ms");
  }

  if (clignotement.estEcoulee()) {
    ledBleue.basculer();
  }
}
```

3. Fonctions intégrées — `millis()`

```ino
// Étape 3.1 — Clignotement non bloquant avec millis() (encapsulé dans
la classe Minuterie)
#include <Arduino.h>
#include "Led.h"
#include "Minuterie.h"

namespace {
constexpr unsigned long INTERVALLE_MS = 1000;
Led ledBleue(LED_BUILTIN);
Minuterie minuterie(INTERVALLE_MS);
}  // namespace

void setup() {
```

```
  ledBleue.begin();
}

void loop() {
  if (minuterie.estEcoulee()) {
    ledBleue.basculer();
  }
  // loop() n'est jamais bloquée : on pourrait faire d'autres tâches
ici
}
```

- `millis()` renvoie le nombre de millisecondes écoulées depuis le démarrage. La valeur est un `unsigned long` sur 32 bits, comptée par l'interruption du Timer0.
- `previousMillis` mémorise l'instant du dernier changement d'état. Le mot-clé `static` conserve sa valeur entre deux appels de `loop()`.
- Quand `currentMillis - previousMillis >= interval`, on bascule la LED avec `!digitalRead(...)` et on met `previousMillis` à jour.

Question : quel est l'avantage par rapport à `delay()` ? `delay()` **bloque** le microcontrôleur, qui ne peut rien faire d'autre (lire un capteur, répondre au port série…). Avec `millis()`, `loop()` tourne en permanence et ne fait que *vérifier* si l'intervalle est écoulé. Le programme reste **non bloquant**, ce qui permet de gérer plusieurs tâches avec des périodes différentes.

Remarque : `millis()` revient à 0 après 2^{32} ms, soit environ 49,7 jours. L'écriture `currentMillis - previousMillis` en arithmétique non signée reste juste même au moment de ce débordement. Écrire `currentMillis >= previousMillis + interval` serait faux à ce moment-là.

4. Acquisition des données des capteurs

Bibliothèques installées : **arduino-sht** (Sensirion, v2.0.0), **Adafruit LTR329 and LTR303** (v2.0.1, avec Adafruit BusIO en dépendance), **KXTJ3-1057** (v0.1.1) et **FastLED** (v3.10.7).

Les trois capteurs partagent le **même bus I2C** (fils SDA/SCL). Chacun y est identifié par son **adresse** : SHTC3 = 0x70, LTR-303 = 0x29, KXTJ3-1057 = 0x0E.

4.2 — SHTC3 (température / humidité)

```
// Étape 4.2 — Capteur de température et d'humidité SHTC3 (9600 bauds)
#include <Arduino.h>
#include "CapteurTempHum.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 9600;
constexpr unsigned long PERIODE_MS = 1000;
CapteurTempHum capteur;
}  // namespace
```

```
void setup() {
  Serial.begin(DEBIT_BAUDS);
  delay(1000);  // laisse le temps au moniteur série de s'ouvrir

  Serial.println(capteur.begin() ? "init(): success" : "init():
failed");
}

void loop() {
  MesureTempHum mesure{};
  if (capteur.lire(mesure)) {
    Serial.println("SHT:");
    Serial.print("  RH: ");
    Serial.println(mesure.humidite, 2);
    Serial.print("  T:  ");
    Serial.println(mesure.temperature, 2);
  } else {
    Serial.println("Error in readSample()");
  }
  delay(PERIODE_MS);
}
```

- `Wire.begin()` initialise l'I2C en maître.
- `sht.init()` sonde le bus, détecte le modèle SHT présent (SHTC3 ici) et l'initialise. Elle renvoie `true` en cas de succès.
- `sht.readSample()` déclenche une mesure, lit les octets bruts et vérifie leur CRC. Elle les convertit ensuite en °C et en %HR.
- `sht.getTemperature()` et `sht.getHumidity()` renvoient les dernières valeurs converties (`float`). Le 2e argument de `Serial.print(val, 2)` fixe le nombre de décimales à 2.
- Sortie attendue à 9600 bauds : `SHT:  RH: 60.81  T: 24.27`, une mesure par seconde.

4.3 — LTR-303A (lumière)

```
// Étape 4.3 — Capteur de lumière LTR-303A (115200 bauds)
#include <Arduino.h>
#include "CapteurLumiere.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 115200;
CapteurLumiere capteur;
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  while (!Serial) {
    delay(10);
  }
  Serial.println("Adafruit LTR-303 simple test");
```

```
  // Gain x1, intégration 50 ms, une mesure toutes les 50 ms (comme
dans le TP)
  if (!capteur.begin(LTR3XX_INTEGTIME_50, LTR3XX_MEASRATE_50)) {
    Serial.println("Couldn't find LTR sensor!");
    for (;;) {
      delay(10);
    }
  }
  Serial.println("Found LTR sensor!");
}

void loop() {
  MesureLumiere mesure{};
  if (capteur.nouvelleDonnee() && capteur.lire(mesure)) {
    Serial.print("CH0 Visible + IR: ");
    Serial.print(mesure.visibleIr);
    Serial.print("\t\tCH1 Infrared: ");
    Serial.print(mesure.infrarouge);
    Serial.print("\t\tLux: ");
    Serial.println(mesure.lux, 1);
  }
  delay(1000);
}
```

- `setGain(LTR3XX_GAIN_1)` règle le gain à ×1, la plage la plus large (jusqu'à environ 64 klux). `setIntegrationTime(50 ms)` fixe la durée de comptage d'une mesure. `setMeasurementRate(50 ms)` fixe la période entre deux mesures.
- `newDataAvailable()` indique si une nouvelle mesure est prête. `readBothChannels(ch0, ch1)` lit le **canal 0 (visible + IR)** et le **canal 1 (IR seul)**.
- La lumière visible correspond à peu près à CH0 − CH1. Ces valeurs sont des **comptes bruts sur 16 bits**, pas des lux. Pour obtenir des lux, la datasheet du LTR-303 donne une formule qui dépend du rapport CH1/(CH0+CH1), du gain et du temps d'intégration. Nous l'avons utilisée dans la partie 5 (méthode `CapteurLumiere::calculerLux()`).
- Le moniteur série doit être réglé sur **115200 bauds**.

4.4 — KXTJ3-1057 (accéléromètre 3 axes)

```
// Étape 4.4 — Accéléromètre KXTJ3-1057 (115200 bauds)
#include <Arduino.h>
#include "Accelerometre.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 115200;
constexpr float FREQUENCE_HZ = 6.25f;  // 0.781 ... 1600 Hz
constexpr uint8_t PLAGE_G = 2;         // ±2, 4, 8 ou 16 g
constexpr bool HAUTE_RESOLUTION = false;

Accelerometre accelerometre(0x0E);
}  // namespace
```

```ino
void setup() {
  Serial.begin(DEBIT_BAUDS);
  delay(100);

  if (accelerometre.begin(FREQUENCE_HZ, PLAGE_G, HAUTE_RESOLUTION)) {
    Serial.println("IMU initialized.");
  } else {
    Serial.println("Failed to initialize IMU.");
    for (;;) {
    }
  }
}

void loop() {
  accelerometre.veille(false);  // sort l'IMU du mode veille
  const Acceleration a = accelerometre.lire();
  accelerometre.veille(true);   // remet l'IMU en veille

  Serial.print(" Acceleration X float = ");
  Serial.println(a.x, 4);
  Serial.print(" Acceleration Y float = ");
  Serial.println(a.y, 4);
  Serial.print(" Acceleration Z float = ");
  Serial.println(a.z, 4);

  delay(1000);
}
```

- Paramètres : 6,25 échantillons/s, plage ±2 g, résolution normale, adresse I2C **0x0E**.
- `standby(false)` sort le capteur de veille pour la mesure, et `standby(true)` l'y remet pour économiser l'énergie.
- `readRegister(&dataLowRes, KXTJ3_ZOUT_H)` lit un registre de 8 bits. `&dataLowRes` est l'**adresse** de la variable : la fonction y écrit la valeur lue (passage par pointeur). Ici, l'appel sert surtout à **tester la communication**, car `axisAccel(Z)` relit elle-même les registres `ZOUT_L/ZOUT_H` sur 16 bits et les convertit en **g**.
- **Observation :** carte posée à plat, on lit X ≈ 0, Y ≈ 0 et **Z ≈ 1 g**. Un accéléromètre mesure l'accélération *propre* : au repos, il mesure la réaction du support à la pesanteur, soit 1 g vers le haut. En inclinant la carte, la gravité se répartit sur X et Y. En la secouant, les valeurs varient brusquement.

4.5 — LEDs RGB (WS2812 × 21)

```ino
// Étape 4.5 — 21 LEDs RGB : effet « cylon » multicolore (57600 bauds)
#include <Arduino.h>
#include "BandeauLed.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 57600;
```

```
constexpr uint8_t FACTEUR_ATTENUATION = 250;  // 250/256 par pas
constexpr unsigned long PAS_MS = 10;

BandeauLed bandeau;
uint8_t teinte = 0;

void allumerEtAttenuer(uint8_t index) {
  bandeau.definir(index, CHSV(teinte++, 255, 255));
  bandeau.afficher();
  bandeau.attenuer(FACTEUR_ATTENUATION);
  delay(PAS_MS);
}
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  Serial.println("resetting");
  bandeau.begin(84);
}

void loop() {
  Serial.print("x");
  for (uint8_t i = 0; i < BandeauLed::NB_LEDS; ++i) {  // aller
    allumerEtAttenuer(i);
  }
  Serial.print("x");
  for (int i = BandeauLed::NB_LEDS - 1; i >= 0; --i) {  // retour
    allumerEtAttenuer(static_cast<uint8_t>(i));
  }
}
```

- `CRGB leds[21]` est le tableau en mémoire : une couleur R, G, B de 3 octets par LED. Rien n'est envoyé aux LEDs avant l'appel à `FastLED.show()`.
- `addLeds<WS2812, DATA_PIN, RGB>` déclare le type de LED et la broche de données (D4). Les WS2812 n'utilisent qu'**un seul fil de données**, donc `CLOCK_PIN` est inutile. `setBrightness(84)` limite la luminosité globale à 84/255, soit environ 33 %.
- `CHSV(hue, 255, 255)` décrit une couleur par sa teinte (*hue*), avec saturation et luminosité maximales. `hue++` fait défiler l'arc-en-ciel.
- `fadeall()` applique `nscale8(250)` à chaque LED, ce qui multiplie chaque composante par 250/256 (≈ 0,977) à chaque pas. Les LEDs déjà allumées s'estompent progressivement et forment une « traînée ». L'aller-retour produit l'effet « cylon » multicolore.

5. Partie non guidée

Choix communs :

- La « luminosité en lux » est calculée par la classe `CapteurLumiere` avec la formule de la datasheet LTR-303 (gain ×1, intégration 100 ms).
- Les capteurs sont lus de façon **non bloquante** avec la classe `Minuterie` (basée sur `millis()`).
- Moniteur série à **115200 bauds**.

I.1 — Seuil de luminosité

Si la luminosité dépasse 500 lux, les 21 LEDs passent au rouge et un message s'affiche. Sinon, elles sont éteintes.

```
// Partie 5.I — Seuil de luminosité : LEDs rouges + message si > 500
lux (115200 bauds)
#include <Arduino.h>
#include "BandeauLed.h"
#include "CapteurLumiere.h"
#include "Minuterie.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 115200;
constexpr float SEUIL_LUX = 500.0f;

BandeauLed bandeau;
CapteurLumiere capteur;
Minuterie periodeMesure(500);
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  bandeau.begin(84);

  if (!capteur.begin()) {
    Serial.println("Capteur LTR-303 introuvable !");
    for (;;) {
      delay(10);
    }
  }
  Serial.println("Seuil de luminosite : pret");
}

void loop() {
  if (!periodeMesure.estEcoulee()) {
    return;
  }

  MesureLumiere mesure{};
  if (!capteur.lire(mesure)) {
    return;
  }
```

```
  Serial.print("Visible+IR = ");
  Serial.print(mesure.visibleIr);
  Serial.print("  IR = ");
  Serial.print(mesure.infrarouge);
  Serial.print("  Lux = ");
  Serial.println(mesure.lux, 1);

  const bool tropLumineux = mesure.lux > SEUIL_LUX;
  if (tropLumineux) {
    Serial.println("Luminosite > 500 lux : LEDs rouges allumees");
  }
  bandeau.remplir(tropLumineux ? CRGB::Red : CRGB::Black);
  bandeau.afficher();
}
```

Pour tester : sous l'éclairage normal d'une salle (≈ 300 lux), les LEDs restent éteintes. En éclairant le capteur avec la lampe d'un téléphone, la valeur dépasse 500 lux, les LEDs deviennent rouges et le message s'affiche.

I.2 — Détection d'inclinaison

```
// Partie 5.I — Détection d'inclinaison (115200 bauds)
// X > 0.5 g -> "Inclinée à droite", X < -0.5 g -> "Inclinée à gauche"
#include <Arduino.h>
#include "Accelerometre.h"
#include "Minuterie.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 115200;
constexpr float SEUIL_G = 0.5f;  // sin(30°) = 0.5 -> seuil d'environ
30°

enum class Inclinaison { Gauche, APlat, Droite };

Accelerometre accelerometre(0x0E);
Minuterie periodeAffichage(500);

Inclinaison determinerInclinaison(float x) {
  if (x > SEUIL_G) {
    return Inclinaison::Droite;
  }
  if (x < -SEUIL_G) {
    return Inclinaison::Gauche;
  }
  return Inclinaison::APlat;
}

const char* texte(Inclinaison inclinaison) {
  switch (inclinaison) {
```

```
    case Inclinaison::Droite: return "Inclinée à droite";
    case Inclinaison::Gauche: return "Inclinée à gauche";
    default:                  return "A plat";
  }
}
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  delay(100);
  // 50 Hz, ±2 g suffit : on mesure la projection de la gravité (|X|
<= 1 g)
  if (!accelerometre.begin(50, 2)) {
    Serial.println("Failed to initialize IMU.");
    for (;;) {
    }
  }
  accelerometre.veille(false);  // accéléromètre actif en continu
  Serial.println("IMU initialized.");
}

void loop() {
  if (!periodeAffichage.estEcoulee()) {
    return;
  }
  const float x = accelerometre.lire().x;

  Serial.print("X = ");
  Serial.print(x, 3);
  Serial.print(" g  -> ");
  Serial.println(texte(determinerInclinaison(x)));
}
```

Au repos, X ≈ 0 g, ce qui affiche « A plat ». Quand on penche la carte d'un angle θ autour de l'axe Y, on mesure X = sin(θ) g. Le seuil de 0,5 g correspond donc à une inclinaison d'environ **30°**. Le signe de X donne le côté (droite ou gauche, selon l'orientation du capteur sur la carte).

II.1 — Station météo embarquée

```
// Partie 5.II — Station météo embarquée (115200 bauds)
// a. Affichage toutes les 5 s
// b. T > 30 °C et HR < 30 % -> "Risque de sécheresse"
// c. Luminosité < 100 lux   -> "Nuit détectée"
#include <Arduino.h>
#include "CapteurTempHum.h"
#include "CapteurLumiere.h"
#include "Minuterie.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 115200;
constexpr unsigned long PERIODE_MS = 5000;
```

```
constexpr float TEMPERATURE_SECHERESSE = 30.0f;
constexpr float HUMIDITE_SECHERESSE = 30.0f;
constexpr float SEUIL_NUIT_LUX = 100.0f;

CapteurTempHum capteurTempHum;
CapteurLumiere capteurLumiere;
Minuterie periodeAffichage(PERIODE_MS);
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  delay(1000);

  if (!capteurTempHum.begin()) {
    Serial.println("SHTC3 : init failed");
  }
  if (!capteurLumiere.begin()) {
    Serial.println("LTR-303 : introuvable");
  }
  Serial.println("Station meteo demarree");
}

void loop() {
  if (!periodeAffichage.estEcoulee()) {
    return;  // non bloquant
  }
  Serial.println("----- Station meteo -----");

  MesureTempHum climat{};
  const bool climatOk = capteurTempHum.lire(climat);
  if (climatOk) {
    Serial.print("Temperature : ");
    Serial.print(climat.temperature, 2);
    Serial.println(" C");
    Serial.print("Humidite    : ");
    Serial.print(climat.humidite, 2);
    Serial.println(" %");
  } else {
    Serial.println("Erreur lecture SHTC3");
  }

  MesureLumiere lumiere{};
  const bool lumiereOk = capteurLumiere.lire(lumiere);
  if (lumiereOk) {
    Serial.print("Luminosite  : ");
    Serial.print(lumiere.lux, 1);
    Serial.println(" lux");
  } else {
    Serial.println("Erreur lecture LTR-303");
  }
```

```
  if (climatOk && climat.temperature > TEMPERATURE_SECHERESSE &&
climat.humidite < HUMIDITE_SECHERESSE) {
    Serial.println("Risque de sécheresse");
  }
  if (lumiereOk && lumiere.lux < SEUIL_NUIT_LUX) {
    Serial.println("Nuit détectée");
  }
}
```

- (a) L'affichage se fait toutes les 5 s grâce à `millis()` (période `PERIODE = 5000`), sans `delay()`.
- (b) « Risque de sécheresse » s'affiche si **T > 30 °C ET HR < 30 %** (opérateur &&).
- (c) « Nuit détectée » s'affiche si la luminosité est **< 100 lux**. Pour tester, il suffit de couvrir le capteur avec un doigt.

II.2 — Simulation d'une alerte IoT

```
// Partie 5.II — Simulation d'une alerte IoT (115200 bauds)
// a. Inclinaison dangereuse : |accélération| > 2 g sur un axe
// b. Surchauffe             : température > 40 °C
// c. Obscurité totale       : luminosité < 10 lux
// Format : "Alerte : [Type d'alerte] à [millis] ms" (émise quand la
condition devient vraie)
#include <Arduino.h>
#include "Accelerometre.h"
#include "CapteurTempHum.h"
#include "CapteurLumiere.h"
#include "Minuterie.h"
#include "Alerte.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 115200;
constexpr float SEUIL_CHOC_G = 2.0f;
constexpr float SEUIL_SURCHAUFFE_C = 40.0f;
constexpr float SEUIL_OBSCURITE_LUX = 10.0f;

Accelerometre accelerometre(0x0E);
CapteurTempHum capteurTempHum;
CapteurLumiere capteurLumiere;

Minuterie periodeAccel(20);     // lecture rapide pour ne pas rater un
choc
Minuterie periodeCapteurs(500);

Alerte alerteInclinaison("Inclinaison dangereuse");
Alerte alerteSurchauffe("Surchauffe");
```

```
Alerte alerteObscurite("Obscurité totale");

bool depasse(const Acceleration& a, float seuil) {
  return fabs(a.x) > seuil || fabs(a.y) > seuil || fabs(a.z) > seuil;
}
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  delay(1000);

  if (!capteurTempHum.begin()) {
    Serial.println("SHTC3 : init failed");
  }
  if (!capteurLumiere.begin(LTR3XX_INTEGTIME_100,
LTR3XX_MEASRATE_100)) {
    Serial.println("LTR-303 : introuvable");
  }
  // Plage ±8 g : avec ±2 g la mesure sature à 2 g et "> 2 g" ne
serait jamais atteint
  if (!accelerometre.begin(100, 8)) {
    Serial.println("KXTJ3 : init failed");
  }
  accelerometre.veille(false);

  Serial.println("Systeme IoT pret");
}

void loop() {
  if (periodeAccel.estEcoulee()) {
    alerteInclinaison.verifier(depasse(accelerometre.lire(),
SEUIL_CHOC_G), Serial);
  }

  if (periodeCapteurs.estEcoulee()) {
    MesureTempHum climat{};
    if (capteurTempHum.lire(climat)) {
      alerteSurchauffe.verifier(climat.temperature >
SEUIL_SURCHAUFFE_C, Serial);
    }
    MesureLumiere lumiere{};
    if (capteurLumiere.lire(lumiere)) {
      alerteObscurite.verifier(lumiere.lux < SEUIL_OBSCURITE_LUX,
Serial);
    }
  }
}
```

- Format respecté : `Alerte : Surchauffe à 15200 ms`. L'heure est la valeur de `millis()` au moment de la détection.

- **Plage de l'accéléromètre portée à ±8 g.** Avec la plage par défaut de ±2 g, la mesure **sature** à 2 g et la condition « > 2 g » ne serait jamais vraie.
- On compare la **valeur absolue** (`fabs`) pour détecter les deux sens sur chaque axe.
- Chaque alerte n'est émise **qu'au moment où la condition devient vraie** (détection de front montant dans la classe `Alerte`, qui mémorise l'état précédent). Le moniteur n'est donc pas inondé tant que la condition persiste, comme le ferait un vrai objet IoT qui limite ses envois.
- L'accéléromètre est lu toutes les 20 ms pour ne pas rater un choc bref. La température et la lumière sont lues toutes les 500 ms.

II.3 — Clignotement synchronisé avec l'accélération

```
// Partie 5.II — Clignotement des LEDs RGB selon l'accélération
(115200 bauds)
// X > 1 g -> rouge, Y > 1 g -> vert, Z > 1 g -> bleu (les couleurs
s'additionnent)
#include <Arduino.h>
#include "Accelerometre.h"
#include "BandeauLed.h"
#include "Minuterie.h"

namespace {
constexpr unsigned long DEBIT_BAUDS = 115200;
constexpr float SEUIL_G = 1.0f;
constexpr unsigned long DEMI_PERIODE_MS = 200;  // 200 ms allumé / 200
ms éteint

Accelerometre accelerometre(0x0E);
BandeauLed bandeau;
Minuterie clignotement(DEMI_PERIODE_MS);
bool allume = false;

CRGB couleurPour(const Acceleration& a) {
  CRGB couleur = CRGB::Black;
  if (a.x > SEUIL_G) couleur += CRGB::Red;
  if (a.y > SEUIL_G) couleur += CRGB::Green;
  if (a.z > SEUIL_G) couleur += CRGB::Blue;
  return couleur;
}
}  // namespace

void setup() {
  Serial.begin(DEBIT_BAUDS);
  bandeau.begin(84);

  // Plage ±4 g pour mesurer au-delà de 1 g sans saturer
  if (!accelerometre.begin(50, 4)) {
    Serial.println("Failed to initialize IMU.");
    for (;;) {
```

```
    }
  }
  accelerometre.veille(false);
  Serial.println("IMU initialized.");
}

void loop() {
  if (!clignotement.estEcoulee()) {
    return;
  }
  const Acceleration a = accelerometre.lire();
  allume = !allume;

  bandeau.remplir(allume ? couleurPour(a) : CRGB(CRGB::Black));
  bandeau.afficher();

  Serial.print("X=");
  Serial.print(a.x, 2);
  Serial.print(" Y=");
  Serial.print(a.y, 2);
  Serial.print(" Z=");
  Serial.println(a.z, 2);
}
```

- X > 1 g donne du rouge, Y > 1 g du vert et Z > 1 g du bleu. Si plusieurs axes dépassent le seuil, les couleurs **s'additionnent** (par exemple rouge + bleu = magenta).
- Le clignotement (200 ms allumé, 200 ms éteint) est géré par `millis()`.
- La plage est réglée à ±4 g pour mesurer au-delà de 1 g sans saturer.
- **Remarque :** à plat, Z vaut environ 1 g à cause de la gravité. Avec le bruit de mesure, Z oscille autour du seuil et la LED bleue peut clignoter de façon irrégulière au repos. Pour éviter cela, on pourrait utiliser un seuil un peu plus haut (1,2 g) ou soustraire la gravité.

Conclusion

Ce TP nous a permis de :

- mettre en place toute la chaîne de développement de la carte UCA (IDE, package de carte, pilote USB et bibliothèques) ;
- comprendre la structure `setup()` / `loop()` ;
- utiliser l'UART en sortie (`Serial.print`) et en entrée (`Serial.available` / `Serial.read`, conversion ASCII) ;
- remplacer `delay()` par `millis()` pour écrire des programmes non bloquants ;
- exploiter les trois capteurs I2C et les LEDs RGB.

La partie non guidée montre comment combiner ces briques en petites applications, comme une station météo ou des alertes de type IoT. Elle a aussi mis en évidence deux points qui ne

sont pas dans l'énoncé : la **saturation** de l'accéléromètre (choix de la plage de mesure) et la différence entre **comptes bruts et lux** pour le capteur de lumière.

Annexe — Résultats de compilation

Cible : `rfthings-avr:avr:uca`, version 3.9+ (ATmega328PB), package RFThings AVR Boards 0.1.1-rc6. Mémoire disponible : 30 720 octets de Flash et 2 048 octets de RAM.

Projet	Flash (octets)	RAM (octets)
Etape_1_6_Blink	1246 (4 %)	11 (0 %)
Etape_2_1_Sortie_Serie_Blink_Mod	2522 (8 %)	208 (10 %)
Etape_2_2_Entree_Serie_1_chiffre	2686 (8 %)	282 (13 %)
Etape_2_2_Entree_Serie_plusieurs_chiffres	2714 (8 %)	261 (12 %)
Etape_3_1_Millis	1182 (3 %)	19 (0 %)
Etape_4_2_UCA_Board_SHTC3	8954 (29 %)	570 (27 %)
Etape_4_3_UCA_Board_LTR303	10182 (33 %)	554 (27 %)
Etape_4_4_UCA_Board_KXTJ3	8286 (26 %)	543 (26 %)
Etape_4_5_UCA_Board_RGBLED	4790 (15 %)	374 (18 %)
Partie_5_II_1_Station_Meteo	12970 (42 %)	746 (36 %)
Partie_5_II_2_Alerte_IoT	14536 (47 %)	682 (33 %)
Partie_5_II_3_Clignotement_Accelerometre	11554 (37 %)	661 (32 %)
Partie_5_I_1_Seuil_Luminosite	12874 (41 %)	751 (36 %)
Partie_5_I_2_Detection_Inclinaison	8304 (27 %)	535 (26 %)

Compilation avec `--warnings all` : **aucun avertissement** dans notre code.